App Dev Project Report

1. My Details

Name: Adith N Poojary
LinkedIn: www.linkedin.com/in/adith-n-poojary-56433b340/
Email: npoojaryadith@gmail.com

---

2. Project Details

Project Title: Trekking Management Application

Problem Statement:

To design and build a multi-role web application that streamlines the **booking, management, and staffing of trekking expeditions**. The system enables users to discover and book treks, provides assigned staff with participant rosters and schedules, and allows administrators to oversee operations, staff approvals, and slot capacities through real-time metrics.

Approach:

The application was developed using **Python and Flask** for backend routing and business logic, integrated with an **SQLite relational database** created programmatically.

- **Architecture & UI:** Server-side rendered using **Jinja2 templating, HTML5, CSS3, and Bootstrap 5**, delivering a responsive dashboard layout tailored to three distinct user roles (Admin, Staff, and User).
- **User Management & Operations:** Users can browse available expeditions, view trek itineraries, reserve slots, and track booking history.
- **Administrative Workflows:** Administrators manage trek listings, approve/reject staff requests, dynamically assign guides to upcoming expeditions, and monitor overall platform statistics via dedicated overview metrics.
- **Data Integrity:** All core business constraints—such as slot decrements, multiple bookings management, and automated staff availability tracking—are enforced directly through structured SQL queries on the backend without relying on client-side script execution.

---

3. AI/LLM Declaration

I used **Gemini** to assist in formatting responsive HTML/Jinja2 templates using Bootstrap for the application's user interface.

The extent of AI/LLM usage is around **2-5 %**, limited to **UI layout generation.**

All final implementation logic, debugging, and integration were done manually.

---

4. Technologies and Frameworks Used

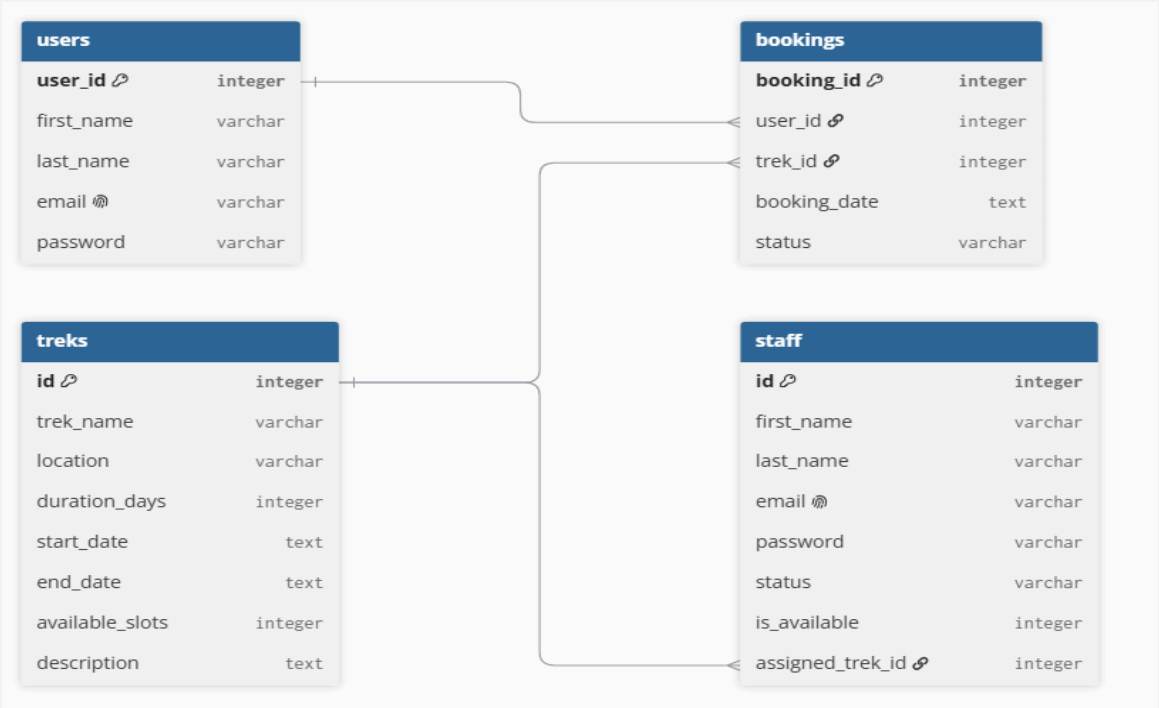
Technology / Library	Purpose
Flask	Core Python backend web framework for handling routing, request processing, and application logic.
SQLite (sqlite3)	Lightweight, file-based relational database used programmatically for data persistence and queries.
Jinja2	Server-side templating engine for rendering dynamic data, loops, and conditional elements in HTML.
HTML5 & CSS3	Structural markup and custom stylesheet formatting for interface layout and styling.
Bootstrap 5	CSS framework used for responsive grid systems, cards, forms, tables, and UI components.
Bootstrap Icons	Icon font library providing visual indicators for sidebar navigation and dashboard metrics.

---

5. Database Schema / ER Diagram

Tables:

1. **User (`users`)** — stores registered user details (`user_id`, `first_name`, `last_name`, `email`, `password`).
2. **Staff (`staff`)** — stores staff accounts, approval status, availability, and trek assignments (`id`, `first_name`, `last_name`, `email`, `password`, `status`, `is_available`, `assigned_trek_id`).
3. **Trek (`treks`)** — stores expedition listings, schedules, capacity, and route descriptions (`id`, `trek_name`, `location`, `duration_days`, `start_date`, `end_date`, `available_slots`, `description`).
4. **Booking (`bookings`)** — records reservations made by users for specific treks (`booking_id`, `user_id`, `trek_id`, `booking_date`, `status`).



Relationships:

- **One-to-Many** → `users` → `bookings` *(A single user can book multiple treks)*
- **One-to-Many** → `treks` → `bookings` *(A single trek can have multiple user reservations)*
- **One-to-Many** → `treks` → `staff` *(A trek can be assigned one or more staff guides)*

---

6. API Resource Endpoints

Endpoint	Method	Description
/signup	GET, POST	Render registration page / Register new user or submit staff request
/signin	GET, POST	Render sign-in page / Authenticate credentials and establish session
/admin/dashboard	GET	Render admin overview statistics (users, staff, bookings, treks)
/admin/treks	GET	View list of all scheduled treks and assigned guides
/admin/add_trek	POST	Create and store a new trek expedition in the database
/admin/staff	GET	View categorized lists of pending, approved, and blacklisted staff
/admin/staff/action/<staff_id>/<action>	POST	Update staff status (approve or blacklist)
/admin/staff/assign	POST	Assign an approved staff guide to an upcoming trek

Endpoint	Method	Description
/admin/users	GET	View registered users list sorted by total bookings
/staff/dashboard	GET	View treks assigned to the logged-in staff member
/staff/manage/<trek_id>	GET	View trek details and participant roster for assigned staff
/staff/profile	GET, POST	View and update staff profile information
/user/dashboard	GET	View current and past bookings for the logged-in user
/user/browse	GET	Browse available upcoming treks and assigned staff guides
/user/book_trek	POST	Book a selected trek and decrement available slot capacity
/user/history	GET	View complete booking history and expedition dates
/user/profile	GET, POST	View and update user personal profile details
/logout	GET	Clear user/staff session and redirect to sign-in page

7. Architecture and Features

Architecture Overview:

- `app.py` — Main Flask application entry point managing routing, database execution, session handling, and application logic.
- `models.py` — Database initialization script containing SQL schema definitions and table creation logic for SQLite.
- `/static` — Static asset directory containing custom stylesheets (`main.css`) for registration, authentication, and layout styling.
- `/templates` — Jinja2 HTML templates providing server-side rendered dynamic views across Admin, Staff, and User interfaces.

Implemented Features:

- **Role-Based Authentication & Access Control:** Separate registration and login workflows for users (trekkers), staff guides, and platform administrators.
- **Dynamic Trek Discovery & Booking:** Trekkers can browse active expeditions, inspect assigned guide information, reserve slots with automatic capacity decrement, and review historical bookings.
- **Staff Guide Workflow & Itinerary Management:** Guides can view assigned treks, inspect upcoming schedules, and access live participant rosters for booked users.
- **Administrative Control & Staff Governance:** Admins can create new expeditions, approve or blacklist staff applicants, dynamically assign guides to upcoming treks, and track registered user engagement.
- **Real-Time Operational Metrics:** Admin dashboard displaying live counts for registered users, active staff, total bookings, and available treks.
- **Profile Management:** In-app personal credential and profile updating for both trekkers and staff members.

Additional Features:

- **Automated Staff Availability Tracking:** Database-driven availability checks that free assigned staff when an expedition's end date passes.
- **Dynamic Slot & Capacity Constraints:** Server-side slot validation preventing overbooking when expedition capacity reaches zero.

---

8. Video Presentation

Drive Link:

Video Presentation.mp4